BombLab 实验报告

一．实验目标与完成情况

本实验通过分析个人 bomb 的 x86-64 汇编，将各关对输入的检查恢复为随 challenge 变化的求解函数，并在 defuser.c 中实现。正式六关分别涉及函数调用、数组递推、跳转表、递归、链表和函数指针；Honor 还需要结合前六关产生的 context 构造第七行输入。

统一公开测试结果为正式六关 60/60、Honor 10/10；公开列表以外的 seed 314159 完整运行也通过了六关和 Honor。

二．实验环境与分析方法

实验通过 VS Code SSH 连接的 Linux x86_64 环境中完成，使用 GCC、GNU Make、Python 3、GNU GDB 15.1 和 objdump。个人炸弹导入后，在仓库根目录编译 defuser。正式求解代码集中在 defuser.c，接口保持不变。

先查看 phase_01 至 phase_06 的汇编，通过 valid_u32_fields 的参数和 sscanf 返回值检查确定整数个数，再追踪 challenge 的寄存器数据流。对辅助函数逐个反汇编；跳转表、函数指针表及链表节点则用 GDB 读取。最后把计算关系写成 C，并分别运行公开测试和额外 seed 验证。

下文以 C 表达式描述运算，^ 表示按位异或。U32(x) 表示取低 32 位。除明确使用 64 位 challenge 的移位外，结果均按 uint32_t 运算，加法和乘法按模 2^32 回绕，右移为逻辑右移。

三．环境问题与处理

最初在远程 Linux 中使用了 Mac 桌面的 /Users 路径，导致文件不存在；将附件上传到远程仓库后改用相对路径。导入时脚本用 r:gz 强制按 gzip 解包，但下载的文件能以 r:* 正常打开，且包含 bomb 和 package.json，因此将读取模式改为自动识别后成功导入。该修改只涉及领取工具，不属于解法。

四．第一关：函数调用与参数传递

第一关要求输入一个无符号整数，关键是追踪 p1_compute 调用三个辅助函数时的参数和返回值。将 challenge 的低 32 位、高 32 位和右移 16 位后的低 32 位分别记为 L、H、M，可以恢复出 s1=(3×H)^(M+0x7a5b)，s0=4×L+(H^0x9581)，最终答案为 (s0+8×s1)^0x9bfd。其中需要注意，p1_step1 没有修改 edi，因此后续传给 p1_step0 的参数仍然包含 H。实现使用 uint32_t 保持与汇编一致的 32 位回绕行为，公开测试 10/10，并通过额外 seed 314159 的验证。

```
root@ics:~/ics-2026-bomblab# make && python3 public_test.py --phase 1
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challe
nge.c cli.c -o defuser
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Public tests: 10/10
```

第一关公开测试



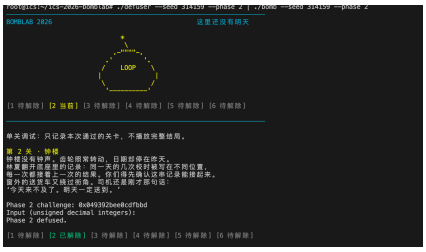
第一关额外 seed 314159 验证

五．第二关：数组与递推

第二关要求输入六个无符号整数，主要考查数组和循环递推。程序首先检查第一个数是否等于 (challenge>>36)&63，随后用循环检查后五个数。根据循环中移位量每次增加 3、数组下标每次增加 1 的规律，可以恢复出 v[i]=2×v[i−1]+(((challenge>>(3×i+3))&63)^i)，其中 i 从 1 到 5。实现时依次生成六个数，先完成截取和异或，再与前一个数的两倍相加。公开测试 10/10，并通过额外 seed 314159 的验证。

```
root@ics:~/ics-2026-bomblab# make && python3 public_test.py --phase 2
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challe
nge.c cli.c -o defuser
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Public tests: 10/10
```

第二关公开测试



第二关额外 seed 314159 验证

六．第三关：分支与跳转表

第三关要求输入两个无符号整数，主要考查 switch 分支和跳转表。第一个数为 challenge&7，用来选择八个分支之一；第二个数的计算参数为 x=U32(challenge>>32)^U32(challenge>>3)。通过读取跳转表并追踪各目标地址，可以确定编号 0～7 实际对应 p3_case_5、p3_case_6、p3_case_0、p3_case_3、p3_case_2、p3_case_4、p3_case_1 和 p3_case_7。随后将各分支中的乘法、加法、移位和异或恢复为 C 表达式，用 switch 生成结果。关键是按照跳转表建立映射，不能直接把输入编号当作函数编号。公开测试 10/10，并通过额外 seed 314159 的验证。

```
root@ics:~/ics-2026-bomblab# make && python3 public_test.py --phase 3
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challe
nge.c cli.c -o defuser
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Public tests: 10/10
```

第三关公开测试



第三关额外 seed 314159 验证

七．第四关：递归与返回值

第四关要求输入一个无符号整数，核心是恢复二分查找式递归及其回溯计算。程序以 challenge&127 为目标，在区间 [0,127] 中递归查找，并使用 k=U32(challenge>>7)^U32(challenge>>32) 参与计算。每次取中点 m，目标等于中点时返回 (m+0xf5)^k；目标较小时递归左区间，返回 (5×子结果)^(m+k)^4；目标较大时递归右区间，最后的异或常量改为 0xb。因此答案不仅与最终找到的位置有关，还取决于整个递归路径。实现保留了递归与回溯顺序，公开测试 10/10，并通过额外 seed 314159 的验证。

```
root@ics:~/ics-2026-bomblab# make && python3 public_test.py --phase 4
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challe
nge.c cli.c -o defuser
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Public tests: 10/10
```

第四关公开测试



第四关额外 seed 314159 验证

八．第五关：结构体与链表遍历：

第五关要求输入两个无符号整数，主要考查结构体布局和链表遍历。根据内存访问偏移，可以确定每个节点包含 next 指针、节点标识和节点数值。程序用 challenge&7 选择起点，用 (challenge>>3)&3 确定后续移动次数，因此实际访问节点数为移动次数加一。

累计值初始为 U32(challenge>>6)^U32(challenge>>32)^0x7569，每访问一个节点便更新为((累计值+节点数值)×5)^节点标识。最终输出累计值和最后访问节点的标识。读取链表后，将所需节点按链序保存为数组，以相同顺序完成计算。公开测试 10/10，并通过额外 seed 314159 的验证。

```
root@ics:~/ics-2026-bomblab# make && python3 public_test.py --phase 5
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challe
nge.c cli.c -o defuser
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Public tests: 10/10
```

第五关公开测试



第五关额外 seed 314159 验证

九．第六关：函数指针与间接调用

第六关要求输入一个无符号整数，主要考查函数指针表和间接调用。程序先计算 x=U32(challenge>>6)^U32(challenge>>32)，再用 challenge&7 选择第一次操作，将结果异或 0xf3b5 后，用 (challenge>>3)&7 选择第二次操作，最后加上 0xf631 得到答案。读取函数指针表可知，下标 0～7 分别对应 op_2、op_0、op_0、op_4、op_5、op_5、op_1 和 op_3。各操作包含加法、异或、移位与循环移位，其中循环移位必须保留从另一端补回的位。实现按照函数表顺序还原两次调用，公开测试 10/10，并通过额外 seed 314159 的验证。

```
root@ics:~/ics-2026-bomblab# make && python3 public_test.py --phase 6
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challe
nge.c cli.c -o defuser
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 10/10
```

第六关公开测试



第六关额外 seed 314159 验证

十．Honor ：入口条件与完整检验

Honor 需要在六关通过后提供第七行输入，关键是同时满足入口布局与数值校验。分析 secret_gate 可知，输入长度须为 36～163 字节，偏移 32 处必须是 OPEN，偏移 36 处开始解析三个无符号整数，因此构造为 32 个 A、OPEN 和三个整数。令 C 为 Honor challenge、K 为 context，先计算 f=K&3、a=U32(C)^U32(C>>32)、b=U32(K)^U32(K>>32)，再计算 s=((5×(a^b)+0xe9d6)^0xa696)^(f×0xe9d6)^U32(K>>16)，以及 t=s^(b+f+0xa696+(a>>1))，输出的三个数依次为 f、s、t。最终使用 snprintf 生成完整输入并检查截断，Honor 公开测试 10/10，额外 seed 314159 的完整运行也显示 Secret phase defused。

```
root@ics:~/ics-2026-bomblab# make && python3 public_test.py --secret
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challe
nge.c cli.c -o defuser
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

Honor 公开测试 10/10

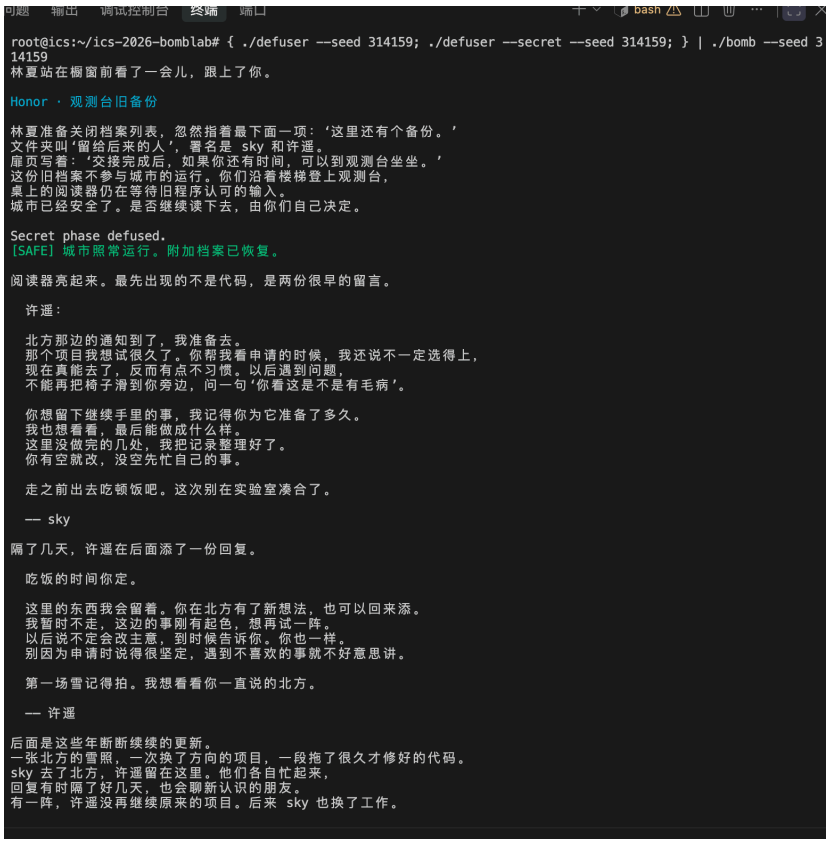
十一. 完整验证与实验结果

正式六关分别使用 python3 public_test.py --phase N 验证，Honor 使用 python3 public_test.py --secret 验证。公开 seed 为 0、1、7、42、99、123、2026、65535、987654321、18446744073709551615。完整 make test 输出六关 60/60、Honor 10/10。

```
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

统一公开测试结果

额外 seed 314159 不在上述公开列表中，完整运行命令如下。输出先后包含六关成功提示和 Secret phase defused，验证了前六关产生的 context 与 Honor 求解结果在同一次运行中一致。



额外 seed 314159 的 Honor 成功结果

十二.实验检查与总结

正式关卡统一通过 write_answer 将 uint32_t 数组格式化为一行十进制口令，函数成功返回 0。Honor 单独使用 snprintf，并检查错误和截断。解法不依赖运行时读取 bomb，也不调用外部程序；从二进制读取的常量、链表数据和函数表映射已写入计算逻辑。

主要需要注意的地方包括寄存器跨调用后的实际值、跳转表顺序、32 位回绕、循环移位以及链表起点与遍历次数。第一关要追踪未被修改的 edi；第三关不能按函数名编号直接分支；第五关包含起点处理，因此访问次数比后续步数多一次；第六关函数表字节偏移需要转换为元素下标。

当前实现完成了六关和 Honor，并通过已有公开测试与 seed 314159 的额外验证。